A REPORT

ON

"Performance analysis of edge detection algorithm between GP and GPU"

Submitted to

INTEGRATED TEST RANGE, DEFIANCE RESEARCH AND DEVELOPMENT ORGANIZATION

BY

Gopikishan Mahto

Graduate Summer Tranee

UNDER THE GUIDANCE OF

Mr Amit Sardar

Scientist –'E'

Mr. Gopikishan Mahto

Summer Tranee

Integrated Test Range

Defence Research and Development Organisation Chandipur, Balasore, Odisha

ACKNOWLEDGEMENTS

I am grateful to many people who have given their time and guidance in completing this project. First, I would like to extend my sincere gratitude to H. K. Ratha (Sc-'G'), the Director of Integrated Test Range, Chandipur. I am also thankful to HRD group for selecting and placing me in Directorate of Computer and Data Processing, which is having the best of computer facilities.

I am very grateful to Mr. Rakesh Barua (Sc-'F'), GD (Computer) for assigning me the required resources and giving me encouragement and guidance.

I am very thankful to Mr. Amit Sardar (Sc-'E') for assigning me a very interesting project and giving me the support and technical guidance. I am also indebted to all employees of Directorate of Computer and Data Processing for their generosity in helping me. I am also grateful to officer in-charge of Knowledge Center for permitting me to use rich library facilities. At last, I would like to thank all people of this establishment.

Gopikishan Mahto

ITR, DRDO

Department of Computer and Data Prosseing Division
Chandipur and Balasore Odisha 756025

CERTIFICATE

This is certify that the project entitled

"**Performance analysis of edge detection algorithm between GP and GPU**"

Submitted by

Gopikishan Mahto

is a record of bonafide work carried out by him. This project is the outcome of train-ing undergone at the Directorate of Computer and Data Processing Division, Inte-grated Test Range, Defence Research and Development Organization, Chandipur, Balasore, Odisha during the period of May-June 2022, under my guidance and su-pervision

Date:

Mr. Amit Sardar (Sc-'E')
Project Guide

ITR, DRDO

Department of Computer and Data Prosseing Division
Chandipur and Balasore Odisha 756025

CERTIFICATE

This is certify that the project entitled

"**Performance analysis of edge detection algorithm between GP and GPU**"

Submitted by

Gopikishan Mahto

is a record of bonafide work carried out by him. This project is the outcome of train-ing undergone at the Directorate of Computer and Data Processing Division, Inte-grated Test Range, Defence Research and Development Organization, Chandipur, Balasore, Odisha during the period of May-June 2022, under my guidance and su-pervision

Date:

Mr. Amit Sardar (Sc-'E')
Project Guide

DECLARATION

I hereby declare that the project entitled "**Performance analysis of edge detection algorithm between GP and GPU**"has been submitted for the graduate summer tranee at Computer and Data Processing Division, ITR, Chandipur.

Gopikishan Mahto

Overview

Main goal of this project is to implement image morphology algorithms in GPU and make analysis of its performance in comparison with standard library OpenCV.

Introduction

A central processing unit (CPU) is designed to handle complex tasks, such as time slicing, virtual machine emulation, complex control flows and branching, security, etc. In contrast, graphical processing unites (GPUs) only do one thing well. They handle billions of repetitive low level tasks.

Originally designed for the rendering of triangles in 3D graphics, they have thousands of arithmetic logic units (ALUs) compared with traditional CPUs that commonly have only 4 or 8. Many types of scientific algorithms spend most of their time doing just what GPUs are good for: performing billions of repetitive arithmetic operations.

The reason behind the discrepancy in floating-point capability between the CPU and the GPU is that the GPU is specialized for compute-intensive, highly parallel computation - exactly what graphics rendering is about - and therefore designed such that more transistors are devoted to data processing rather than data caching and flow control.

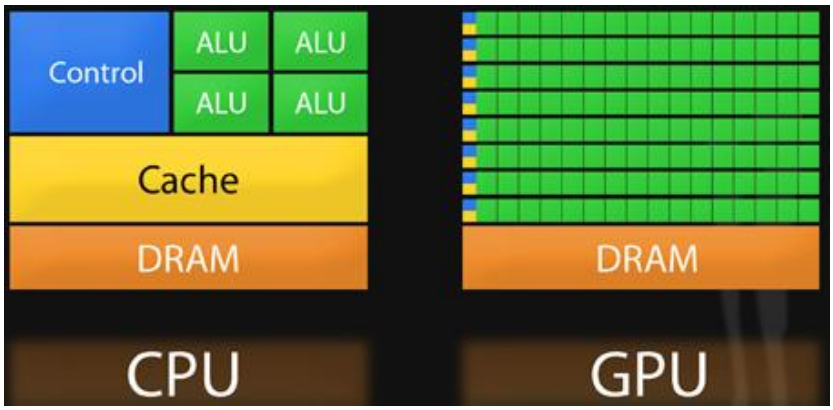


Fig. CPU vs GPU architecture

Data-parallel processing maps data elements to parallel processing threads

Many applications that process large data sets can use a data-parallel programming model to speed up the computations. In 3D rendering, large sets of pixels and vertices are mapped to parallel threads. Similarly, image and media processing applications such as post-processing of rendered images, video encoding and decoding, image scaling, stereo vision, and pattern recognition can map image blocks and pixels to parallel processing threads. In fact, many algorithms outside the field of image rendering and processing are accelerated by data-parallel processing, from general signal processing or physics simulation to computational finance or computational biology.

The advent of multicore CPUs and manycore GPUs means that mainstream processor chips are now parallel systems. Furthermore, their parallelism continues to scale with Moore's law. The challenge is to develop application software that transparently scales its parallelism to leverage the increasing number of processor cores, much as 3D graphics applications transparently scale their parallelism to manycore GPUs with widely varying numbers of cores.

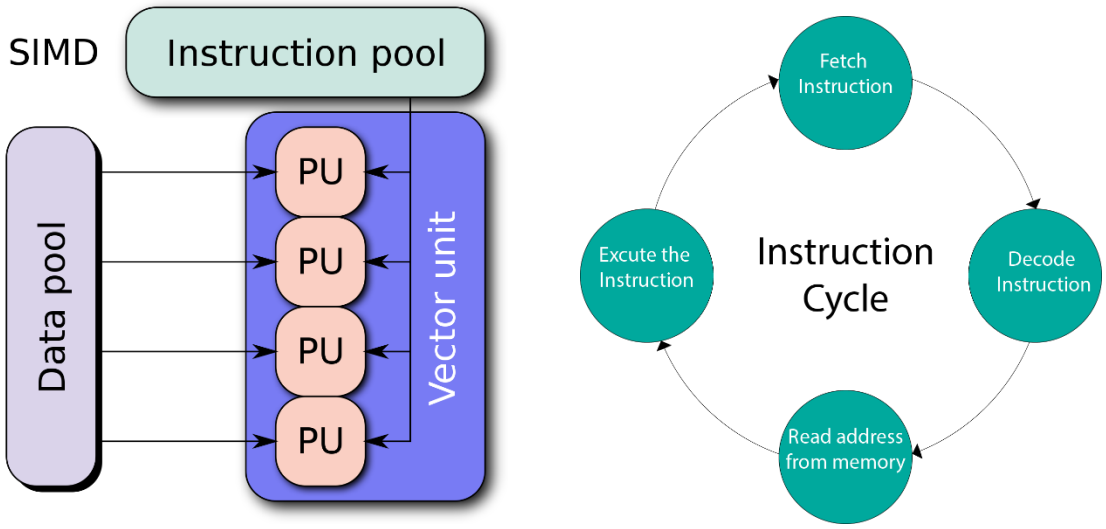


Fig. GPU vs CPU instruction Cycle

Programming

When computer scientists first attempted to use GPUs for scientific computing, the scientific codes had to be mapped onto the matrix operations for manipulating traingles. This was incredibly difficult to do, and took a lot of time and dedication. However, there are now high level languages (such as CUDA and OpenCL) that target the GPUs directly, so GPU programming is rapidly becoming mainstream in the scientific community.

A GPU program comprises two parts: a *host* part the runs on the CPU and one or more *kernels* that run on the GPU. Typically, the CPU portion of the program is used to set up the parameters and data for the computation, while the kernel portion performs the actual computation. In some cases the CPU portion may comprise a parallel program that performs message passing operations using MPI.

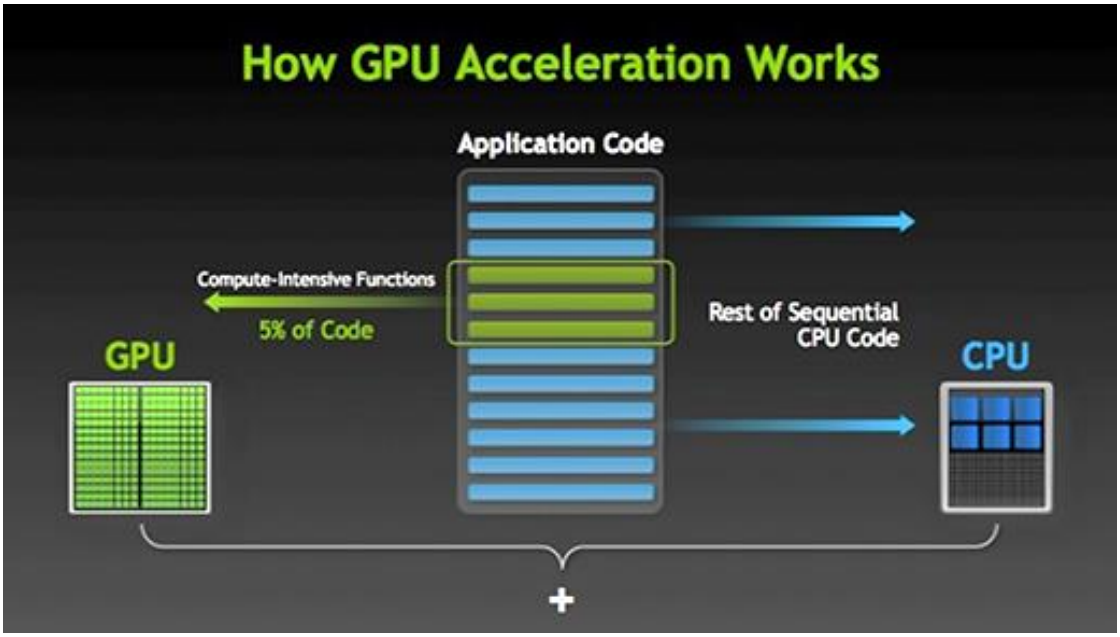


Fig. GPU Acceleration

CUDA

CUDA stands for Compute Unified Device Architecture. It is an extension of C/C++ programming. CUDA is a programming language that uses the Graphical Processing Unit (GPU). It is a parallel computing platform and an API (Application Programming Interface) model, Compute Unified Device Architecture was developed by Nvidia. This allows computations to be performed in parallel while providing well-formed speed. Using CUDA, one can harness the power of the Nvidia GPU to perform common computing tasks, such as processing matrices and other linear algebra operations, rather than simply performing graphical calculations.

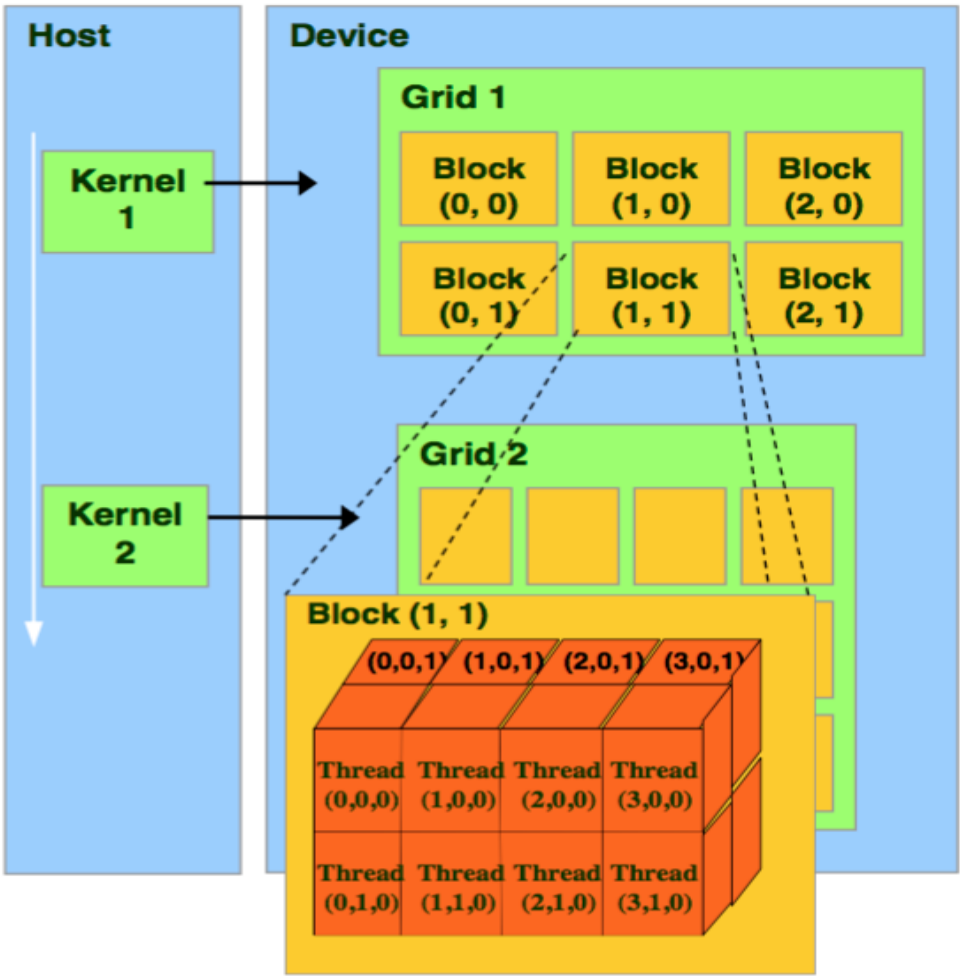


Fig. CUDA Structure

Why do we need CUDA?

- GPUs are designed to perform high-speed parallel computations to display graphics such as games.
- Use available CUDA resources. More than 100 million GPUs are already deployed.
- It provides 30-100x speed-up over other microprocessors for some applications.
- GPUs have very small Arithmetic Logic Units (ALUs) compared to the somewhat larger CPUs. This allows for many parallel calculations, such as calculating the color for each pixel on the screen, etc.

Architecture of CUDA

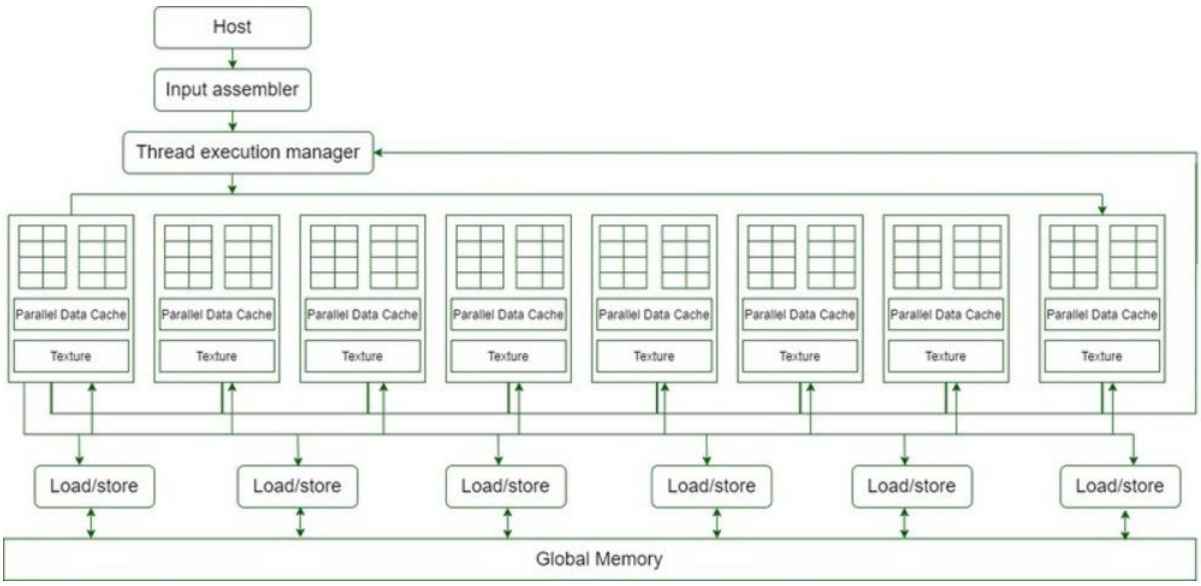


Fig. Architecture of CUDA

- 16 Streaming Multiprocessor (SM) diagrams are shown in the above diagram.
- Each Streaming Multiprocessor has 8 Streaming Processors (SP) ie, we get a total of 128 Streaming Processors (SPs).
- Now, each Streaming processor has a MAD unit (Multiplication and Addition Unit) and an additional MU (multiplication unit).
- The GT200 has 240 Streaming Processors (SPs), and more than 1 TFLOP processing power.
- Each Streaming Processor is gracefully threaded and can run thousands of threads per application.
- The G80 card supports 768 threads per Streaming Multiprocessor (note: not per SP).
- Eventually, after each Streaming Multiprocessor has 8 SPs, each SP supports a maximal of 96 threads. Total threads that can run – **128 * 96 = 12,228 times.**
- Therefore, these processors are called **massively parallel.**
- The G80 chips have a memory bandwidth of 86.4GB/s.
- It also has an 8GB/s communication channel with the CPU (4GB/s for uploading to the CPU RAM, and 4GB/s for downloading from the CPU RAM).

How CUDA works?

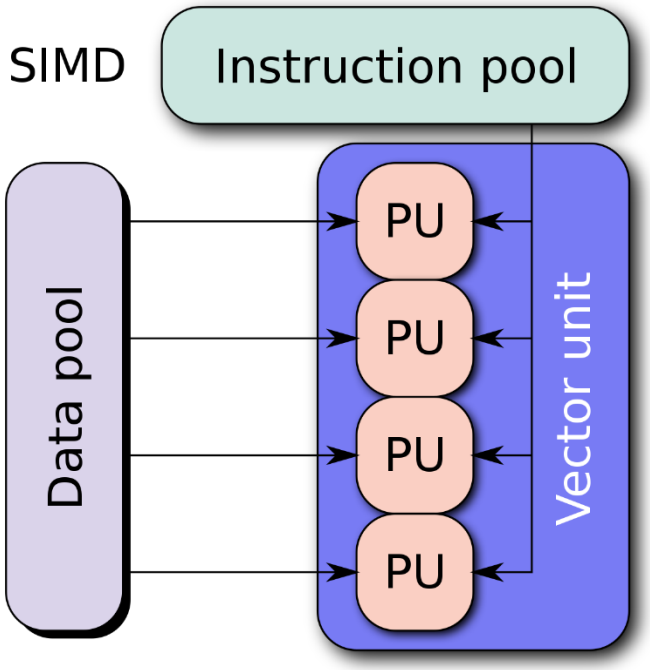


Fig. Working of CUDA

- GPUs run one kernel (a group of tasks) at a time.
- Each kernel consists of blocks, which are independent groups of ALUs.
- Each block contains threads, which are levels of computation.
- The threads in each block typically work together to calculate a value.
- Threads in the same block can share memory.
- In CUDA, sending information from the CPU to the GPU is often the most typical part of the computation.
- For each thread, local memory is the fastest, followed by shared memory, global, static, and texture memory the slowest.

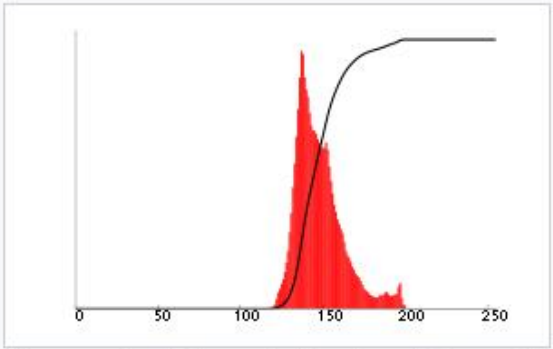
Algorithm

1. Histogram Equalization

Histogram

Histogram is a graphical representation of the intensity distribution of an image. In simple terms, it represents the number of pixels for each intensity value considered.

Before Histogram Equalization



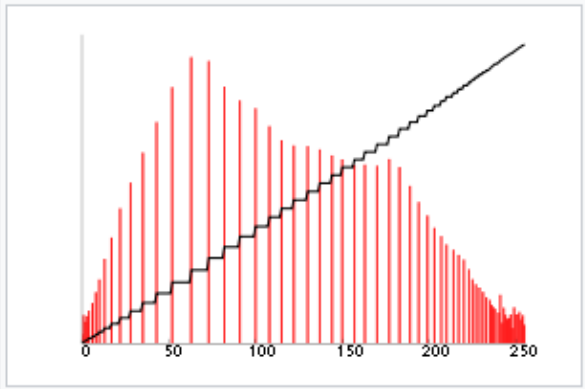
Corresponding histogram (red) and cumulative histogram (black)

In the above figure, X-axis represents the tonal scale (black at the left and white at the right), and Y-axis represents the number of pixels in an image. Here, the histogram shows the number of pixels for each brightness level (from black to white), and when there are more pixels, the peak at the certain brightness level is higher.

Histogram Equalization

Histogram Equalization is a computer image processing technique used to improve contrast in images. It accomplishes this by effectively spreading out the most frequent intensity values, i.e. stretching out the intensity range of the image. This method usually increases the global contrast of images when its usable data is represented by close contrast values. This allows for areas of lower local contrast to gain a higher contrast.

After Histogram Equalization



Corresponding histogram (red) and cumulative histogram (black)

Steps involved

- Generate histogram from given image
- Calculate new histogram data
- Apply the histogram equalization on each partition

Algorithm in CPU :

```
# To generate histogram data

image = gray scale (original image)

histogram_data = array.all_zeros[256]

for row in image :

    for col in image :

        histogram_data[row][col] +=1


# To generate new histogram data

cummulative_freq = 0

new_histogram_data = array.all_zeros[256]

for val in range(0 to 256) :

    cummulative_freq = histogram_data[row][col]

    new_histogram_data[row][col] = round(cummulative_freq * 255) / total_pixel )


# To write new hisogram data in our image

for row in image :

    for col in image :

        image[row][col] =new_histogram_data[image[row][col[]
```

Algorithm in GPU :

```
# Defining cuda kernels
```

```
@cuda.jit
def func1(img_gray, cuda_hist):
    row , col = cuda.grid(2)
    shared_arr = cuda.shared.array(shape = 256 ,dtype = int32)
    if cuda.threadIdx.x < 256  and cuda.threadIdx.y == 0:
        shared_arr[cuda.threadIdx.x] = 0
    cuda.syncthreads()
    if row < img_gray.shape[0] and col < img_gray.shape[1] :
        cuda.atomic.add(shared_arr , img_gray[row][col] ,1)
    cuda.syncthreads()
    if cuda.threadIdx.x < 256 and cuda.threadIdx.y == 0:
        cuda.atomic.add(cuda_hist , cuda.threadIdx.x , shared_arr[cuda.threadIdx.x])


@cuda.jit
def func2(total_pixel,hist , new_hist):
    curr = 0
    for i in range(0,256):
        curr += hist[i]
        new_hist[i] = round((curr * 255 )/total_pixel)


@cuda.jit
def func3(cuda_img_gray ,  cuda_hist_new):
    row , col = cuda.grid(2)
    if row < img_gray.shape[0] and col < img_gray.shape[1] :
        cuda_img_gray[row][col] =  cuda_hist_new[cuda_img_gray[row][col]]
```

```
# Transfer data to GPU

cuda_img_gray = cuda.to_device(img_gray)

cuda_img_gray = cuda.to_device(img_gray)

hist_type = [0] * 255

cuda_hist_new = cuda.to_device(hist_type)


# Invoke kernels from cpu code

func1[(blocks_x,blocks_y),threads](cuda_img_gray , cuda_hist)

func2[1,1](total_pixel , cuda_hist , cuda_hist_new)

func3[(blocks_x,blocks_y),threads](cuda_img_gray  , cuda_hist_new)


# Data back to CPU

img_gray_new = cuda_img_gray.copy_to_host()
```

2. Top hat morphology

In morphology and digital image processing, top-hat and black-hat transform are operations that are used to extract small elements and details from given images. These two types of transforms in which, the top-hat transform is defined as the difference between the input image and its opening by some structuring element, while the black-hat transform is defined as the difference between the closing and the input image. These transforms are used for various image processing tasks, such as feature extraction, background equalization, image enhancement, and others.

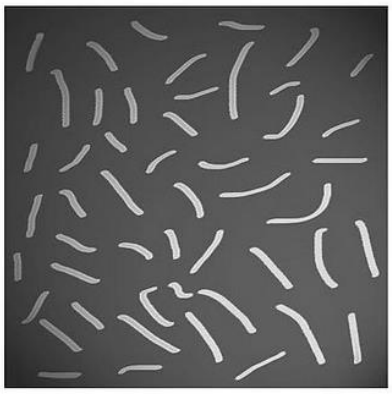
Here we will use Opening and Closing morphological operations.

White Top Hat Morphology

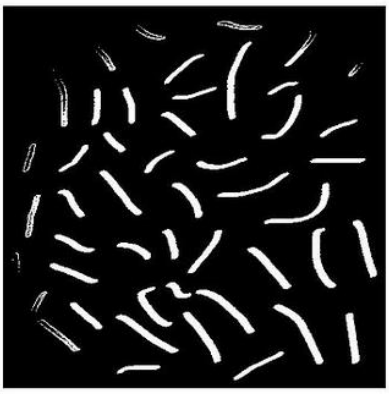
image opening = erosion followed by dilation on gray scale image

White top hat = image – image opening

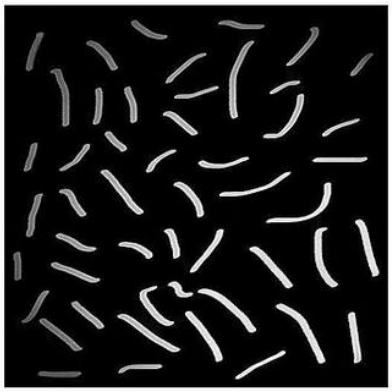
The top-hat filter is used to enhance bright objects of interest in a dark background. The black-hat operation is used to do the opposite, enhance dark objects of interest in a bright background.



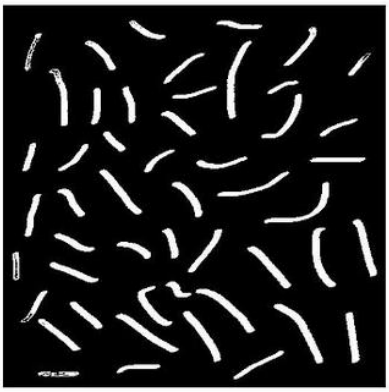
Nonuniform lighting condition input image



Threshold a nonuniform lighting input image



Top-hat transform applied to input image



Threshold image after top-hat transform applied

```
#defining GPU kernels

@cuda.jit

def erosion(cuda_img_new ,cuda_img , cuda_k) :

    row , col = cuda.grid(2)

    min = 256

    if row < cuda_img.shape[0] - cuda_k[2] and row > cuda_k[2] -1 and col < cuda_img.shape[1] -
cuda_k[2] and col > cuda_k[2] -1  :
```

```
        for i in range(0,cuda_k[0]) :
            for j in range(0 , cuda_k[1]) :
                val = cuda_img[row-cuda_k[2]+i][col-cuda_k[2]+j]
                if val < min :
                    min = val
        cuda_img_new[row][col] = min


@cuda.jit
def reset_0(cuda_img , cuda_k) :
    row , col = row , col = cuda.grid(2)
    if ( row > cuda_img.shape[0] - cuda_k[2] -1 or row < cuda_k[2] )  :
        cuda_img[row][col] = 0
    if (col > cuda_img.shape[1] - cuda_k[2] -1 or col < cuda_k[2] ) :
        cuda_img[row][col] = 0


@cuda.jit
def dal(cuda_img_new ,cuda_img , cuda_k) :
    row , col = cuda.grid(2)
    max = 0
    if row < cuda_img.shape[0] - cuda_k[2]  and row > cuda_k[2] -1 and col < cuda_img.shape[1] -
cuda_k[2] and col > cuda_k[2] -1  :
        for i in range(0,cuda_k[0]) :
            for j in range(0 , cuda_k[1]) :
                val = cuda_img[row-cuda_k[2]+i][col-cuda_k[2]+j]
                if val > max :
                    max = val
```

```
        cuda_img_new[row][col] = max


@cuda.jit

def w_top_hat(orig_img, opening , cuda_k) :

    row , col = cuda.grid(2)

    if row < opening.shape[0] - cuda_k[2] and row > cuda_k[2] -1 and col < opening.shape[1] - cuda_k[2]
and col > cuda_k[2] -1  :

        orig_img[row][col] = orig_img[row][col] - opening[row][col]


# Executing kernels from CPU

image =  gray scale ( original image )

img_gray_pad = pad image with required pixels as per kernel size effect with all values to  256 # for
erosion

cuda_img_gray = cuda.to_device(img_gray_pad)

ero[blocks, threads](cuda_ero_img ,cuda_img_gray , cuda_k)

reset_0[blocks, threads](cuda_ero_img , cuda_k)

dal[blocks, threads](cuda_ero_dal_img ,cuda_ero_img , cuda_k)

w_top_hat[blocks, threads](cuda_img_gray , cuda_ero_dal_img , cuda_k)

w_top_h = cuda_img_gray.copy_to_host()

w_top_h = trim image(w_top_h) as per kernel size
```

Result and analysis

- CPU : i5 10th gen
- Ram : 8gb 2300ghz
- GPU : Nvidia mx 250
- 720p Video was used to test

- frame shape (720 * 1280)
- Gray scaling of image was done using opencv for execution of histogram equilization

Run type	fps
Running in CPU using function	2
Running in CPU using function with JIT	50
Running in CPU using openCV library	75

Run type	fps	threads	Blocks Image / threads
Running in GPU using cuda kernel function	65	256 , 4	3 , 320
Running in GPU using cuda kernel function	60	256 , 3	3 , 427
Running in GPU using cuda kernel function	50	256 , 2	3 , 640
Running in GPU using cuda kernel function	45	256 , 1	3 , 1280

Table. Histogram result

Conclusion and Future scope

- GPU should be used for multiple set of work.
- In single work CPU is faster because of data transfer for a single work is not optimal approach.
- In 1st iteration GPU takes more time but after that it is much faster then CPU.
- We can work on a good GPU which has more no of cores.
- Optimization can be done on algorithm.

- Data transfer can be minimized between host and device.

Reference

- Massively parallel programming (edition 3)
- YouTube
- Lectures and notes of Wen-mei W. Hwu